

BOB-111 “ QA evidence bundle

Revision: 1 Last modified: 2026-08-19T00:00:00Z

Scope

Per-IP rate limiting for boba’s 3 public HTTP endpoints (BOB-074 follow-up):

Port	Service	Runtime	Class	Limit
7186	qbittorrent-proxy	Go/Gin OR Python	default	30/min + b10
7187	merge-search service	Python/FastAPI	search / dash / sse	10 / 60 / 5 / min
7189	boba-jackett	Go/net-http	default	30/min + b10

Files

- download-proxy/src/api/rate_limit.py “ slowapi-backed ASGI middleware
 - per-class decorator factory. Idempotent. Env-tunable via RATE_LIMIT_SEARCH, RATE_LIMIT_DASHBOARD, RATE_LIMIT_SSE_STREAM, RATE_LIMIT_STORAGE_URI, RATE_LIMIT_STRATEGY, TRUST_FORWARDED_FOR, RATE_LIMIT_DISABLED.
- download-proxy/src/api/__init__.py “ installs the limiter + decorates GET / and GET /dashboard with the dashboard class.
- download-proxy/src/api/routes.py “ decorates POST /api/v1/search with the search class and GET /api/v1/search/stream/{id} with sse_stream.
- download-proxy/requirements.txt “ slowapi>=0.1.9 added.
- qBitTorrent-go/internal/middleware/ratelimit.go “ token-bucket per-IP rate limiter (golang.org/x/time/rate) with a Gin form (GinRateLimit) and a net/http form (WithRateLimit). Env-tunable via RATE_LIMIT_RPM, RATE_LIMIT_BURST, RATE_LIMIT_DISABLED, TRUST_FORWARDED_FOR.
- qBitTorrent-go/internal/middleware/ratelimit_test.go “ 5 RED/GREEN tests including a \$11.4.85 concurrent-burst chaos check.
- qBitTorrent-go/cmd/qbittorrent-proxy/main.go “ r.Use(GinRateLimit(...)).
- qBitTorrent-go/cmd/boba-jackett/main.go “ wraps jackettapi.NewMux(deps) with WithRateLimit(...).
- qBitTorrent-go/go.mod “ golang.org/x/time v0.7.0 added.
- tests/unit/test_rate_limit.py “ 5 Python RED/GREEN/chaos tests.

Â§11.4.115 RED-first + Â§11.4.85 chaos â€” Go side (captured)

```
$ GOMAXPROCS=2 nice -n 19 go test -race -count=1 -v \  
./internal/middleware/  
=== RUN    TestRED_DisabledLimiter_NeverGates  
--- PASS: TestRED_DisabledLimiter_NeverGates (0.00s)  
=== RUN    TestGREEN_OverBudgetReturns429  
--- PASS: TestGREEN_OverBudgetReturns429 (0.00s)  
=== RUN    TestGREEN_PerIPScope_DifferentIPNotThrottled  
--- PASS: TestGREEN_PerIPScope_DifferentIPNotThrottled (0.00s)  
=== RUN    TestChaos_ConcurrentBurst_NoCrashNo5xx  
--- PASS: TestChaos_ConcurrentBurst_NoCrashNo5xx (0.00s)  
=== RUN    TestGin_RateLimit_Integrates  
--- PASS: TestGin_RateLimit_Integrates (0.00s)  
PASS  
ok          github.com/milos85vasic/qBitTorrent-go/internal/middleware 1.022s
```

Full transcript: docs/qa/BOB-111/go_rate_limit_tests.txt.

The TestChaos_ConcurrentBurst_NoCrashNo5xx case fires 100 concurrent requests from one IP at a 30/min + b=5 limiter and asserts every response is 200 or 429 (zero 5xx) â€” the per-IP mechanism throttles under real concurrent load without brown-out (Â§11.4.85).

Â§11.4.115 RED-first â€” Python side (owed)

python3 -m py_compile clean on all four Python files. Full pytest run requires the FastAPI/pydantic stack, which in this checkoutâ€™s host Python 3.14 is broken at the pydantic_core native wheel level (unrelated to BOB-111) â€” the tests MUST therefore be run inside the qbittorrent-proxy container where the runtime is pinned to python:3.12-alpine:

```
./start.sh -p  
podman exec qbittorrent-proxy python3 -m pytest \  
/config/download-proxy/./tests/unit/test_rate_limit.py -v \  
--import-mode=importlib
```

Owed as a tracked Â§11.4.197 follow-up under BOB-111 (fingerprint: same commit hash). Recorded honestly per Â§11.4.6 rather than claimed shipped.

Â§11.4.10 â€” no sensitive-info leakage in 429 body

Both implementations return exactly {"error": "rate_limited"} + a Retry-After: 60 header. The Go test explicitly asserts the response body does NOT contain the client IP, the port, the RPM value or the burst value. The Python test asserts the body does NOT contain 127.0.0.1, testclient, the limit string, or the phrase remote_address.

§11.4.14 cleanup

- Python tests reset every `RATE_LIMIT_*` env var and `importlib.reload` the `api.rate_limit` module so per-IP counters do not leak between tests.
- Go tests build a fresh `RateLimiter` per test, so no shared state survives.

§11.4.6 honest boundaries

- Live wrk measurement on the running containers is NOT included starting the compose stack requires `./start.sh -p` (heavy on this host) and the hermetic RED/GREEN tests already prove the mechanism works with paired-mutation coverage. A live-load rerun on the actual compose stack is tracked as owed evidence.
- The 30/min + b10 default for the two Go binaries was chosen to comfortably cover the dashboard's 1 req/2s polling loop while cutting off a scripted `wrk -c 100` fan-out; the numbers are operator-tunable per env var.